Data Structures & Algorithms (CSC-214) **Semester: Spring 2025**
Lab 12: Recursion **Date: 2nd May 2025**

Q1: Ackermann's Function: Ackermann's Function is a recursive mathematical algorithm that can be used to test how well a computer performs recursion. Write a function A(m, n) that solves Ackermann's Function. Use the following logic in your function:

If m = 0 then return n + 1

If n = 0 then return A(m−1, 1)

Otherwise, return A(m−1, A(m, n−1))

Test your function in a driver program with: A(3, 2)

Q2: countChars: Count the number of times a certain character appears in a string. Use recursion in the function to traverse the string. Demonstrate the function in a driver program.

Q3: Palindrome Detector: A palindrome is any word, phrase, or sentence that reads the same forward and backward. Here are some well-known palindromes:

Able was I, ere I saw Elba

A man, a plan, a canal, Panama

Desserts, I stressed

Kayak

Write a bool function that uses recursion to determine if a string argument is a palindrome. The function should return true if the argument reads the same forward and backward. Demonstrate the function in a program.

Q4: Linked List: Dry run the following and determine the output.

```cpp
struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

void printList(Node* head) {
    if (head == nullptr) return; // base case
    cout << head->data << " ";
    printList(head->next); // recursive case
}

int main() {
    Node* head = new Node(1);
    head->next = new Node(2);
    head->next->next = new Node(3);
    printList(head);
}
```

Q5: Linked List: Dry run the following code and determine the output. Assume the operation is performed on the linked list created in the above question.

```cpp
Node* reverseList(Node* head) {
    if (head == nullptr || head->next == nullptr)
        return head;
    Node* rest = reverseList(head->next);
    head->next->next = head;
    head->next = nullptr;
    return rest;
}
```

Q6: countNodes: Add a member function named countNode to a singly linked list. The function should return the total number of nodes in the list. Use recursion in the function to traverse the list. Demonstrate the function in a driver program.

Q7: printReverse: Add a member function named printReverse to a singly linked list. The function should print the linked list in reverse. Use recursion in the function to traverse the list. Demonstrate the function in a driver program.

Q8: maxNode Function: Add a member function named maxNode to a singly linked list. The function should return the largest value stored in the list. Use recursion in the function to traverse the list. Demonstrate the function in a driver program.

Submit: your .cpp file along with a screenshot of the complete call stack of any one of the function in the debug mode